

Teleprompter Script

AI-Driven Malware Detection via Windows Event Log Engineering
in Smart Grid Control Systems

Target runtime: 15 minutes · 16 slides

Contents

Slide 1 — Title	2
Slide 2 — Agenda	2
Slide 3 — The Security Gap in Smart Grid Hosts	2
Slide 4 — Smart Grid IT/OT Architecture	2
Slide 5 — Prior Work and Research Gap	2
Slide 6 — Threat Model: Adversary and Goals	3
Slide 7 — Threat Model: MITRE ATT&CK ICS Patterns	3
Slide 8 — Synthetic Dataset: Justification and Statistics	3
Slide 9 — Data Pipeline: Generation to Evaluation	3
Slide 10 — Session-Level Feature Engineering	4
Slide 11 — Classification Results	4
Slide 12 — ROC Curve and Feature Importances	4
Slide 13 — Ablation Study	4
Slide 14 — Conclusion and Future Work	5
Slide 15 — References	5
Slide 16 — Questions	5

Slide 1 — Title

≈0:30

Good morning, and welcome. Today I am presenting our framework for artificial intelligence-driven malware detection in smart grid control systems, using Windows Event Log engineering as the data source. This work sits at the intersection of industrial control system security and machine learning, targeting a layer of the system that has been mandated by regulation but never actually used for automated intrusion detection. *[click to next slide]*

Slide 2 — Agenda

≈0:30

We have six sections. Motivation — why this problem matters. Background — where existing work falls short. Threat model — what adversary we assume. Data and features — how we generate and represent the data. Results — model performance and ablation. And a brief conclusion. *[click to next slide]*

Slide 3 — The Security Gap in Smart Grid Hosts

≈1:15

Modern smart grids are no longer air-gapped. They integrate IP-connected Windows workstations running SCADA software, human-machine interfaces, remote terminal units, and historian servers within the same network fabric as corporate IT. This convergence has been exploited in some of the most consequential cyber attacks of the last fifteen years. Stuxnet in 2010 targeted Windows hosts embedded in nuclear centrifuge control loops. The 2015 and 2016 Ukrainian grid attacks combined spear phishing with lateral movement through the IT/OT boundary to cause physical power outages. Colonial Pipeline in 2021 showed that IT-side ransomware could force a pipeline operator to shut down OT systems out of caution. *[click]*

The regulatory response has been thorough. NERC CIP-007 mandates event logging on all applicable cyber assets. IEC 62351 specifies audit trail requirements for power system communications. Utilities are legally required to collect Windows Event Logs from their SCADA hosts. *[click]*

The problem is that these logs are collected for compliance and then ignored from a security monitoring perspective. That gap is what we close. *[click to next slide]*

Slide 4 — Smart Grid IT/OT Architecture

≈0:30

[show figure] This diagram shows the Purdue model architecture we target. Windows hosts appear at all three levels — Level 3 corporate IT, Level 2 SCADA operator stations, and Level 1 RTU and HMI devices. Our pipeline ingests Windows Event Logs from hosts at all three levels and classifies sessions centrally. *[click to next slide]*

Slide 5 — Prior Work and Research Gap

≈1:30

Three research streams are directly relevant to our work, and each one leaves a gap we fill.

The first is log anomaly detection. Tools like Drain, DeepLog, and LogAnomaly have achieved impressive results on distributed system logs — HDFS, Linux syslog, OpenStack. But none of them were designed for Windows Security Event IDs, which carry structured semantics about authentication, process creation, and object access. They also do not model ICS-specific threat patterns.

The second stream is ICS intrusion detection. Carcano and colleagues used critical-state analysis on SCADA command sequences; Goldenberg and Wool built finite automata over Modbus/TCP traffic; the SWaT dataset from Singapore provides the community's main ICS benchmark. But all of these work at the network or physical process level — none look at Windows host audit logs.

The third stream is Windows malware detection on IT endpoints — recent work from 2024 and 2025 uses Sysmon logs and graph-based provenance on corporate machines. The gap is operational context: OT systems run legacy Windows versions with fundamentally different software baselines and event patterns. *[pause]*

Mitchell and Chen confirmed in a 2014 ACM Computing Surveys paper that host-level intrusion detection is rare in the cyber-physical systems literature. That finding still holds today. Our framework is, to our knowledge, the first to specifically target Windows Event Logs at ICS Purdue Levels one through three. *[click to next slide]*

Slide 6 — Threat Model: Adversary and Goals

≈0:45

Our adversary model is grounded in MITRE ATT&CK for ICS. We assume the attacker has already established a foothold on a Level 3 IT host — through spear phishing or a supply-chain compromise — and possesses valid or cracked domain credentials. They have network connectivity across the IT/OT boundary through a misconfigured DMZ or a hijacked remote-access tunnel.

The attack chain has three stages: establish persistence on OT hosts, escalate privileges to system or domain-administrator level, and execute a disruptive payload — either injecting rogue commands into a PLC or denying operator visibility by disrupting the historian. *[click to next slide]*

Slide 7 — Threat Model: MITRE ATT&CK ICS Patterns

≈1:00

We model four specific attack categories, each producing a distinct signature in the Windows Event Log.

Lateral movement: the attacker brute-forces credentials, generating 3 to 8 consecutive Event 4625 logon failures from an unexpected source IP before achieving a 4624 success. Persistence: a 7045 service installation event on a SCADA host, followed by 7036 state change and 4688 process creation. Privilege escalation: a 4672 special-privileges assignment immediately preceding 4688 with an anomalous binary name such as mimikatz or psexec. Reconnaissance: a burst of 8 to 20 Event 4663 object-access records within 30 seconds on HMI or RTU hosts.

We also include evasive variants — rate-limited lateral movement that spaces failures 60 to 300 seconds apart, and slow reconnaissance with 2 to 9 second burst intervals. These make single-feature detection much harder and produce a more realistic evaluation. *[click to next slide]*

Slide 8 — Synthetic Dataset: Justification and Statistics

≈1:30

Before I describe the generator, I want to address why synthetic data. The answer is straightforward: no publicly available Windows Event Log dataset exists for smart grid SCADA or RTU hosts. Conti and colleagues confirmed this in a comprehensive 2021 survey of ICS security testbeds. Nafees and colleagues attribute the gap to liability constraints and the unwillingness of energy operators to disclose network topology through released logs.

Synthetic generation is the established response. Goh and colleagues built the entire SWaT Secure Water Treatment testbed precisely because no real ICS attack data was available — that dataset is now the standard ICS security benchmark. Chen and colleagues in 2022 synthesised APT-style Windows audit sequences for the same reason. We follow this precedent.

Our dataset contains 6,000 labelled sessions: 5,000 benign and 1,000 malicious, totalling 89,288 individual events at a 5-to-1 class ratio. We use 13 Windows Event IDs and 7 smart grid hostnames following real utility naming conventions. The evasive attack variants — visible in the fourth bullet — are the key design decision that prevents the model from relying on a single trivially separable feature. *[click to next slide]*

Slide 9 — Data Pipeline: Generation to Evaluation

≈0:30

[show figure] This is the full data pipeline. The synthetic generator produces labelled CSV records. The preprocessor groups events into sessions and applies a stratified 70/15/15 train-validation-test split by session ID. The feature engineer extracts the 24-dimensional vector per session. The model trainer runs RandomizedSearchCV with 5-fold stratified cross-validation. And the evaluator reports metrics on the held-out test set. *[click to next slide]*

Slide 10 — Session-Level Feature Engineering

≈1:30

The core of our approach is the session-level feature engineering pipeline. Rather than applying sequence modelling to the full event stream, we segment events into sessions using a 30-minute inactivity timeout and extract a compact 24-dimensional feature vector from each session.

The first group computes occurrence counts for each of the 13 Windows Event IDs plus a total count — 14 dimensions. The second group computes time-delta statistics: mean, standard deviation, minimum, and maximum of inter-event gaps in seconds — 4 dimensions. This is the most discriminative group. Reconnaissance attacks burst 20 object-access events in 30 seconds, producing dramatically different timing profiles than normal operator sessions with minutes between actions. The third group captures unique source IPs, usernames, and hostnames as a lateral-spread indicator. The fourth flags rare event IDs. The fifth computes Shannon entropy over the event-ID distribution.

These 24 features require no scaling, handle mixed types natively in a tree ensemble, and produce sub-second batch inference on commodity hardware — key for SIEM integration. *[click to next slide]*

Slide 11 — Classification Results

≈2:00

We compare Random Forest against five baselines spanning from unsupervised to boosted ensemble. The model was selected for four reasons: it handles heterogeneous feature types without normalisation, the `class_weight=balanced` setting compensates for the 5-to-1 imbalance without oversampling, feature importances support the ablation analysis, and inference is sub-millisecond per session.

Walking through the table: the Isolation Forest, which receives no attack labels, achieves macro F1 of 0.601 — the no-label lower bound for a zero-day scenario. Logistic Regression reaches 0.942 and SVM 0.844, confirming the features are not perfectly linearly separable — consistent with the evasive variants we designed. Decision Tree achieves 0.984. XGBoost is a very strong baseline at 0.998. The Random Forest achieves macro F1 of 0.999 and ROC-AUC of 1.000.

To rule out a lucky data split, we repeated the full pipeline across five independent random seeds. F1 mean is 0.999 with standard deviation 0.001, and AUC is 1.000 with zero standard deviation. All numbers were computed from the actual pipeline output. *[click to next slide]*

Slide 12 — ROC Curve and Feature Importances

≈1:00

[show figure] The left panel shows the ROC curve on the held-out test set. It hugs the top-left corner throughout, confirming the AUC of 1.0. The right panel shows the top-ten feature importances by mean decrease in Gini impurity.

Notice that the five most important features are all timing-related: maximum inter-event gap, gap standard deviation, minimum gap, and mean gap. Object-access count — Event 4663 — is also highly ranked. This ranking directly predicts the ablation result on the next slide: removing timing features should cause the second-largest F1 drop. *[click to next slide]*

Slide 13 — Ablation Study

≈1:30

The ablation study removes one feature group at a time, retrains with identical hyperparameters, and re-evaluates macro F1 on the same held-out test set. All results are computed.

Removing Group 1 — event frequency counts — causes the largest drop: F1 falls from 1.000 to 0.944, a delta of negative 0.056. Removing Group 2 — time-delta statistics — causes the second-largest drop to 0.972. Removing Groups 3 through 5 — cardinality, rare-event flags, and sequence entropy — shows zero marginal impact.

A zero marginal delta does not mean entropy is uninformative. Tested in isolation, entropy achieves F1 of 0.77 — that is real signal. The zero delta means that given event counts and timing, entropy adds nothing

further because those two groups already fully separate benign from malicious sessions on this dataset.

The practical takeaway: if you are deploying to a Windows system where timestamp resolution is coarse — some legacy OT systems log with one-second or one-minute granularity — prioritise event counts. On noisier real logs, Groups 3 through 5 are expected to recover importance. *[click to next slide]*

Slide 14 — Conclusion and Future Work

≈1:00

To summarise. We have presented the first open-source framework for Windows Event Log-based malware detection specifically designed for smart grid industrial control systems at Purdue Levels one through three. The framework achieves macro F1 of 0.999 with standard deviation 0.001 across five independent seeds and ROC-AUC of 1.0. Event counts are the most discriminative feature group; timing statistics are second. Inference runs in under half a second per 900-session batch on a single CPU core, confirming operational feasibility alongside a SIEM.

Future directions include evaluation on real ICS event logs from controlled red-team exercises, extension to multi-class MITRE ATT&CK technique attribution, online learning for concept drift adaptation, and adversarial robustness evaluation. *[click to references, then Q&A]*

Slide 15 — References

silent

[This slide displays automatically. No spoken content required. Remain silent or offer to discuss any reference in detail during Q&A.]

Slide 16 — Questions

open

Thank you for your attention. I am happy to take questions on any aspect of the work — the threat model, feature engineering, the evaluation methodology, or the synthetic data justification. *[pause and make eye contact]*

[If no immediate questions:] Perhaps I can start with the question I am most often asked: does near-perfect accuracy on synthetic data translate to real-world performance? The honest answer is that we do not know yet, and evaluation on real ICS event logs from controlled red-team exercises is our highest-priority future work. The evasive attack variants give us some confidence — single-feature maximum F1 drops to 0.86 with them present — but nothing substitutes for real operational data.